Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la Programación y Computación 2, 2do. Semestre 2026

PROYECTO 3: API DE AUTORIZACIÓN DE FACTURAS

202503481 – Ibolya Orsolya David Cadenas

Resumen

La Superintendencia de Administración Tributaria (SAT) de Guatemala requiere un servicio que, al recibir una solicitud de autorización de un Documento Tributario Electrónico (DTE), valide la información del contribuyente y asigne un número de autorización único por cada día. Este ensayo describe el desarrollo de una solución compuesta por dos aplicaciones en C#: un servicio backend construido con .NET Core que expone una API consumible por protocolo HTTP mediante mensajes en formato JSON, y una aplicación frontend en Razor Pages que simula el envío de solicitudes y la consulta de reportes. El backend valida el dígito verificador de cada Número de Identificación Tributaria (NIT) mediante el algoritmo de módulo 11 indicado por la SAT, confirma que el contribuyente exista en el Registro Tributario Unificado, recalcula el IVA y el total de cada factura, y utiliza expresiones regulares para extraer la fecha y hora útiles de un campo de texto libre. Con esta información genera números de autorización correlativos por día y reportes estadísticos de facturas procesadas, con y sin errores, para un rango de fechas determinado.

Palabras clave

API REST, JSON, expresiones regulares, validación de NIT, C#.

Abstract

Guatemala's Tax Administration Superintendency (SAT) requires a service that, upon receiving an authorization request for an Electronic Tax Document (DTE), validates the taxpayer's information and assigns a unique authorization number for each day. This essay describes the development of a solution made up of two C# applications: a backend service built with .NET Core that exposes an API consumable over the HTTP protocol through JSON messages, and a Razor Pages frontend application that simulates sending requests and querying reports. The backend validates the check digit of each Tax Identification Number (NIT) using the modulus-11 algorithm specified by the SAT, confirms that the taxpayer exists in the Unified Tax Registry, recalculates each invoice's VAT and total, and uses regular expressions to extract the useful date and time from a free-text field. With this information, it generates daily sequential authorization numbers and statistical reports of processed invoices, with and without errors, for a given date range.

Keywords

REST API, JSON, regular expressions, NIT validation, C#.

Introducción

La facturación electrónica obliga a los entes recaudadores a validar en tiempo real la información de cada documento tributario y a emitir una autorización única e irrepetible por transacción. Este ensayo documenta el diseño e implementación de una API en C# que recibe solicitudes de autorización de Documentos Tributarios Electrónicos (DTE) y de actualización del Registro Tributario Unificado (RTU) para la Superintendencia de Administración Tributaria (SAT), valida la información recibida, calcula un número de autorización correlativo por día y genera reportes estadísticos. El propósito es mostrar cómo conceptos como el consumo de servicios HTTP con mensajes JSON, el uso de expresiones regulares para extraer datos de texto libre y la validación de reglas de negocio mediante programación orientada a objetos, desarrollados en el curso de Introducción a la Programación y Computación 2, se aplican a un problema cercano al de un sistema de facturación real.

Desarrollo del tema

a. Contexto y planteamiento del problema

El problema planteado consiste en construir, para la Superintendencia de Administración Tributaria (SAT), un servicio web que reciba solicitudes de autorización de Documentos Tributarios Electrónicos (DTE) y responda con un número de autorización único por cada factura. Cada solicitud identifica al emisor y al receptor mediante su Número de Identificación Tributaria (NIT), indica el valor de la operación, el Impuesto al Valor Agregado (IVA) y el total, además de una referencia interna que debe ser única. El sistema debe validar que ambos NIT existan en el Registro Tributario Unificado (RTU) y que su dígito verificador sea correcto, verificar que el IVA y el total hayan sido calculados correctamente, y asignar un correlativo diario que reinicia en 1 cada día y que se antepone a la fecha para formar el número de autorización. Además, el sistema debe generar reportes de facturas procesadas, con errores y sin errores, para un rango de fechas determinado.

b. Arquitectura cliente-servidor: frontend Razor Pages y API en .NET Core

La interfaz de usuario se implementó como una aplicación web en C# bajo el framework Razor Pages (Frontend), que consume la API a través de la clase ClienteApi, mientras que el servicio (Backend) se desarrolló con ASP.NET Core y expone tres controladores: RtuController (POST api/rtu/actualizar, GET api/rtu/consultar/{nit} y GET api/rtu/cantidad), AutorizacionesController (POST api/autorizaciones/procesar) y ConsultasController (GET api/consultas/contribuyente, api/consultas/iva y api/consultas/rango). Todas las solicitudes se reciben como JsonElement y se deserializan con System.Text.Json, y las respuestas se construyen manualmente con Utf8JsonWriter para controlar con precisión la estructura del JSON de salida. El estado del sistema se centraliza en la clase EstadoSistema, que mantiene cinco estructuras enlazadas propias (ListaContribuyentes, ListaFacturas, ListaErrores, ListaCorrelativos y ListaSolicitudes) y protege cada lectura o modificación con un bloqueo (lock) compartido, de modo que las operaciones concurrentes sobre el estado sean seguras. Aunque en este proyecto sí está permitido el uso de las colecciones nativas de C#, se optó por implementar listas enlazadas simples propias (clases Nodo* y Lista*) para cada entidad del sistema, manteniendo la misma disciplina de diseño que en los proyectos anteriores del curso.

c. Validación de NIT y extracción de datos con expresiones regulares

La validación del NIT se implementó en la clase ValidadorNit: primero una expresión regular (^[0-9]{1,20}[0-9K]$) confirma que el formato sea entre 1 y 20 dígitos seguidos de un dígito verificador o la letra K; luego se recorre el número base de derecha a izquierda multiplicando cada dígito por un factor que inicia en 2, se suman los resultados, se obtiene el módulo 11 de la suma y ese residuo se resta a 11 (nuevamente módulo 11), de forma que un resultado de 10 se traduce en el carácter K. Para extraer la fecha y hora del campo de texto libre (por ejemplo "Guatemala, 15/01/2025 15:25 hrs."), la clase ValidadorFecha utiliza una expresión regular con grupos nombrados (fecha y hora) y aserciones de posición que evitan capturar dígitos parciales, descartando el resto del texto —como el nombre del lugar— sin que esto se considere un error, tal como lo exige el enunciado.

d. Número de autorización, cálculo de IVA/TOTAL y reportes por rango de fechas

Cada solicitud de autorización se valida con la clase ValidadorDTE, que revisa en orden la referencia (no vacía, máximo 40 caracteres y no duplicada contra ListaFacturas), el NIT emisor y receptor (formato válido y existencia en el RTU), que el valor sea positivo con dos decimales, y que el IVA y el total coincidan con los valores recalculados (IVA = REDONDEAR(VALOR × 0.12, 2) y TOTAL = VALOR + IVA), registrando cada inconsistencia como un ErrorFactura con un código de tipo (por ejemplo NIT_EMISOR_INEXISTENTE o IVA_INCORRECTO) dentro de ListaErrores. Cuando el DTE es válido, GestorCorrelativos asigna el número de autorización concatenando la fecha en formato yyyyMMdd con un correlativo de ocho dígitos que se reinicia cada día, protegiendo la operación con un bloqueo para evitar que dos solicitudes concurrentes reciban el mismo correlativo. Finalmente, EstadoSistema persiste las cinco estructuras del sistema en un archivo JSON (Datos/estado.json), escribiendo primero en un archivo temporal antes de reemplazar el definitivo, de modo que los reportes por rango de fechas y las consultas por NIT que expone ConsultasController reflejen siempre el último estado guardado.

Conclusiones

El desarrollo de este proyecto permitió aplicar el consumo y la exposición de servicios HTTP con mensajes JSON, así como el uso de expresiones regulares para el procesamiento de texto, a un problema de validación de reglas de negocio similar al de un sistema de facturación electrónica real. En general, el proyecto evidencia que separar la lógica de negocio de la interfaz de usuario en dos aplicaciones independientes, comunicadas por un protocolo estándar como HTTP, facilita que el servicio pueda ser consumido por distintos clientes —incluyendo herramientas externas como Postman— sin depender de una interfaz específica.

Referencias bibliográficas

Crockford, D. (2006). The application/json media type for javascript object notation (JSON) (RFC 4627). IETF. https://www.rfc-editor.org/rfc/rfc4627

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].

Friedl, J. E. F. (2006). Mastering regular expressions (3rd ed.). O'Reilly Media.

International Organization for Standardization. (2003). Information technology — Security techniques — Check character systems (ISO/IEC 7064:2003).

Microsoft. (2024). ASP.NET Core web API documentation. Microsoft Learn. https://learn.microsoft.com/aspnet/core/web-api/